UAMS Full-Stack Architecture & CS Defense Master Guide

From Absolute First Principles to Advanced Engineering: Explaining Your University ERP Project to Your Academic Supervisor

Project: UAMS | **Candidate Role: Lead Software Engineer**

Stack: Next.js 14/15 + Node.js Express + PostgreSQL (Supabase) | **Architecture: Decoupled 3-Tier Enterprise**

Security: OWASP Top 10 Compliant

1. Full-Stack Development from First Principles (The Zero-Jargon Foundation)

If you have never built a full-stack system before, think of software as a **High-End Restaurant**. Every website on the planet operates exactly like this:

1. **The Frontend (The Dining Table & The Menu):**
   This is everything the user sees and touches in their browser (Chrome, Safari). In UAMS, it is written in *React and Next.js*. Just like the customer reads the menu and presses a call button, a student types their admission number and clicks "Pay Fees via M-Pesa". The frontend does **not** have the bank money, and it cannot directly touch the database. It only creates a friendly visual interface.

2. **The API & HTTP Requests (The Waiter):**
   The waiter takes the customer's order to the kitchen. In web engineering, this order is called an **HTTP Request** (like `POST /api/v1/auth/login` or `GET /api/v1/grades`). The waiter brings back the food in a standard format called **JSON** (JavaScript Object Notation), which looks like formatted text with keys and values.

3. **The Backend (The Chef in the Kitchen):**
   Located in `backend/server.js` (Node.js & Express). The kitchen is private; customers are never allowed inside. The chef verifies if the requested dish is allowed, checks the recipe, enforces rules (e.g. *"Is this lecturer allowed to enter marks for this specific unit?"*), and prepares the meal.

4. **The Database (The Secure Pantry & Freezer):**
   Located in **PostgreSQL (Supabase)**. The pantry stores the raw ingredients (rows of student records, grades, passwords, fee transactions). Even the chef must follow strict pantry security rules (Row Level Security) when fetching or updating records.

> **GOLDEN SOFTWARE ENGINEERING RULE TO QUOTE TO YOUR SUPERVISOR**
>
> **"Never Trust the Client" (Zero-Trust Presentation Layer):** Anything running on the student's browser (HTML/JavaScript) can be manipulated using Developer Tools (F12). Therefore, authentic software engineers *never* rely on frontend checks alone. Every single rule, permission, calculation, and security boundary is enforced strictly on the **Backend and Database layer**.

2. Core Concepts You Must Explain Like an Expert

Technical Term	The Simple Real-World Analogy	What It Does in the UAMS Project
RBAC (Role-Based Access Control)	**Keycard Security Levels** in a hospital: A patient can enter the waiting room, a nurse can enter patient rooms, but only surgeons can enter the operating theater.	Partitions UAMS into 7 tiers: Super Admin, Admin, Registrar, HOD, Lecturer, Finance Officer, and Student. A student cannot see the grade submission button; a lecturer cannot view the financial bank ledgers.
JWT (JSON Web Token)	**A Stamped Wristband at a Festival:** When you show your ID at the gate, security stamps your wrist. For the rest of the night, every stall checks the stamp instead of re-verifying your national ID.	When a user signs in with correct credentials, the server cryptographically signs a digital token containing their `role` and `email`. On subsequent requests, the server verifies this signature in microseconds.
HttpOnly Cookie vs **localStorage**	**Lockbox behind the Hotel Desk** vs **Post-it Note on your Shirt:** A post-it note can be snatched by any passerby (XSS virus). A lockbox can only be handled directly by the hotel staff.	UAMS strictly stores authentication tokens inside `HttpOnly` cookies. Browser JavaScript cannot access `document.cookie`, completely eliminating cross-site scripting (XSS) token theft.
Password Hashing (scrypt) vs Encryption	**A Meat Grinder** vs **A Safe:** You can lock something in a safe and unlock it with a key (Encryption). But once you grind meat into sausage, you can never reconstruct the original cow (One-way Hash).	UAMS never stores readable passwords. We use `scrypt` with cryptographic salts. Even if an attacker steals the entire database, they cannot reverse the hashes back into user passwords.
RLS (Row Level Security)	**Bank Safe Deposit Boxes:** Even though all safety boxes are in the same vault, the bank only allows your key to open your specific drawer.	A PostgreSQL security policy where the database engine itself rejects queries if a student attempts to fetch data belonging to another student ID.
Idempotency	**An Elevator Button:** Pressing the floor 5 button once turns on the light. Pressing it 10 more times does not send 10 elevators or charge you 10 times.	Critical for M-Pesa payments: If Safaricom sends the payment callback three times due to network retries, UAMS credits the student's fee ledger exactly once.

3. End-to-End Walkthrough of UAMS Workflows

Workflow A: Institutional Authentication & Session Issuance

1. **User Action:** User navigates to `/login`, selects their role (e.g. *Registrar*), enters credentials, and clicks "Authenticate & Sign In".
2. **Rate Limiting Protection:** The server checks the IP address against a sliding window rate limiter (max 10 attempts per minute) to stop credential stuffing attacks.
3. **Password Verification:** Server runs `scrypt.verifyPassword()` with timing-safe comparison to prevent timing attacks.
4. **Token Generation & Cookie Dispatch:** The backend signs a JWT payload with HMAC-SHA256 using the 64-character secret key. It issues an `HttpOnly` cookie with `SameSite=Strict`.
5. **Role Redirect:** Next.js reads the user role and automatically directs them to their customized workspace (e.g., `/dashboard/registrar`).

Workflow B: Semester Grading & Senate Approval Pipeline

1. **Lecturer Mark Entry:** The Lecturer opens `/dashboard/grades`, selects their course unit (e.g., *BCS 311: Advanced Database Systems*), and inputs CAT marks (out of 30) and Exam marks (out of 70).
2. **Institutional Policy Calculation:** The system invokes the `institutionalPolicy.ts` engine to compute weighted total scores, assign letter grades (A, B, C, D, E), and grade points (4.0 scale).
3. **Departmental Verification (HOD):** The Head of Department reviews the class grade distribution, checks for anomalies, and signs off.
4. **Senate Approval & Publishing (Registrar):** The Registrar receives departmental grade sheets, validates Senate compliance, and marks the grades as *"Approved"*. Once approved, the record is locked against further editing.
5. **Student Transcript Access:** The student logs into `/dashboard/grades` and generates their verified PDF Semester Transcript with institutional watermark.

Workflow C: M-Pesa Fee Settlement & Exam Clearance Card

1. **STK Push Request:** The Student navigates to `/dashboard/finance`, enters their Kenyan mobile number (`07...` or `254...`) and tuition amount, and clicks "Pay with M-Pesa".
2. **Safaricom Daraja Integration:** The system formats the payload to international standard `254XXXXXXXXX`, creates a Base64 timestamp passkey, and calls the Safaricom Daraja API. A PIN prompt appears instantly on the student's handset.
3. **Asynchronous Webhook Callback:** When the student enters their PIN, Safaricom sends a JSON payload to our callback URL `/api/v1/finance/mpesa/callback`.

4. **Idempotent Ledger Settlement:** The server checks the `MpesaReceiptNumber`. If fresh, it inserts a credit record into the university ledger and sets the student balance to zero.

5. **Automated Exam Card Clearance:** The Exam Clearance Engine evaluates: (1) Fee balance is $\le$ threshold, and (2) Class attendance is $\ge 75\%$. If cleared, the student is permitted to download their official QR-coded **Examination Card**.

4. The Academic Defense: Top 12 Tough Supervisor Quizzes & Exact Answers

Q1: "Explain your system architecture. Why didn't you just write a monolithic app?"

How to think about it: Don't put the cash register, the cooking stove, and the bank vault in the same room. Separate them so if one breaks, the others survive.

Say this to supervisor: "We engineered UAMS on a decoupled 3-tier architecture: Next.js (TypeScript) for high-performance server-side rendering and client interface, a dedicated Express Node.js API server for business validation and integration services, and PostgreSQL for relational persistence. Decoupling ensures separation of concerns, enables horizontal microservice scaling, and guarantees that sensitive database connections are never exposed to client-side runtimes."

Trap to avoid: Don't say "Because Next.js was cool." Supervisors hate trend-chasing; they want architectural justification like scalability, security, and separation of concerns.

Q2: "How did you implement security? What stops a student from hacking their grades?"

How to think about it: The student might alter the grade number on their screen using inspect element, but the server will laugh at them because the server only trusts its own database.

Say this to supervisor: "Security is enforced using Defense-in-Depth following OWASP standards: (1) Authentication uses salted `scrypt` hashing with timing-safe comparisons; (2) Session management uses cryptographically signed JWTs stored exclusively in `HttpOnly` cookies with zero `localStorage` usage to eliminate XSS token exfiltration; (3) Every API route verifies role permissions with server-side RBAC guards; and (4) The database layer enforces PostgreSQL Row-Level Security (RLS) so unauthorized SQL mutations are rejected at the database engine level."

Trap to avoid: Never say "We hid the button on the frontend." Hiding buttons is NOT security; anyone can send a POST request with Postman or Curl.

Q3: "What database did you use, and how is your schema normalized?"

How to think about it: Normalization means we don't repeat data. If a student's phone number changes, we update it in 1 place, not in 50 grade rows.

Say this to supervisor: "We utilized PostgreSQL configured in Third Normal Form (3NF). We separated entities into dedicated relational tables: `users`, `students`, `schools`, `departments`, `courses`, `enrollments`, `grades`, `fee_ledger`, and `audit_logs`. We enforce referential integrity with foreign key constraints, cascading rules, and UUID primary keys to prevent sequential ID harvesting attacks."

Trap to avoid: Don't say "We stored everything in JSON." That defeats the purpose of a relational database. Mention foreign keys, indexes, and constraints.

Q4: "What happens if Safaricom sends an M-Pesa payment callback two or three times?"

How to think about it: If a delivery driver rings your doorbell twice with the same package, you don't pay twice or accept two boxes.

Say this to supervisor: "We engineered our M-Pesa webhook handler with strict *idempotency*. When Safaricom transmits a transaction callback, we extract the unique `MpesaReceiptNumber` and check our database transaction cache. If the receipt has already been processed, the system logs a duplicate webhook notice and returns HTTP 200 without re-crediting the student balance."

Trap to avoid: Saying "That never happens." Telco webhooks retry frequently due to latency. Showing you handled idempotency earns maximum engineering points.

Q5: "Why did you build an Institutional Policy Engine instead of hardcoding grade boundaries?"

How to think about it: If the university senate decides next semester that CATs are 40% instead of 30%, a bad programmer has to rewrite 500 lines of code. A good engineer changes 1 config file.

Say this to supervisor: "In enterprise academic ERPs, policies evolve. Different faculties or programs require different assessment weights (e.g. 30/70 for theory vs 40/60 for lab-intensive courses) and different attendance thresholds (e.g. 75% rule). We encapsulated this in an `Institutional Policy Engine` (`institutionalPolicy.ts`) so grading criteria, GPA thresholds, and exam clearance rules are configurable parameters rather than rigid, brittle constants."

Trap to avoid: Admitting you hardcoded values. Emphasize that your system was specifically refactored to align with supervisor recommendations.

Q6: "How does the rotating QR code attendance prevent cheating (proxy attendance)?"

How to think about it: Google Authenticator changes numbers every 30 seconds. A screenshot taken 2 minutes ago is completely useless.

Say this to supervisor: "Traditional paper roll-calls or static QR codes suffer from proxy attendance where students photograph the code and share it on WhatsApp. Our dynamic attendance projector regenerates QR codes every 15 seconds embedded with a time-bound HMAC cryptographic nonce. A student must physically be in the lecture hall to scan it before the nonce expires."

Trap to avoid: Calling it a regular QR code. Emphasize that it is dynamic, ephemeral, and cryptographically signed.

Q7: "What is the difference between Authentication and Authorization?"

How to think about it: Authentication is showing your passport at border control (proving WHO you are). Authorization is whether you have a work visa to enter the country (what you are ALLOWED to do).

Say this to supervisor: "Authentication verifies identity: *'Are you Dr. Kimani?'* (handled via email, salted scrypt password hashing, and MFA TOTP tokens). Authorization verifies permissions: *'Can Dr. Kimani approve Senate transcripts?'* (handled via our RBAC matrix in `rbac.ts`, which determines that only the Registrar has the `grades:approve` permission)."

Trap to avoid: Mixing up the two terms. This is the most common theoretical test examiners use to weed out students.

Q8: "Why did you choose scrypt over basic MD5 or SHA-256 for passwords?"

How to think about it: SHA-256 is a sports car (super fast). That's great for checksums, but terrible for passwords because a hacker can test 10 billion passwords per second on a GPU. Scrypt is heavy sand that slows down the hacker.

Say this to supervisor: "MD5 and SHA-256 are fast hash functions designed for data integrity, making them vulnerable to modern GPU and ASIC rainbow-table attacks. `scrypt` is a memory-hard key derivation function. By tuning CPU and memory cost parameters (`N=16384, r=8, p=1`), it makes hardware-accelerated brute-force attacks computationally and financially prohibitive."

Trap to avoid: Saying "SHA-256 is secure enough for passwords." In 2026, raw SHA-256 without key stretching is considered an OWASP security violation.

Q9: "How do you handle concurrent hostel bed bookings without double-booking?"

How to think about it: If two people grab the last carton of milk at the supermarket at the exact same second, the barcode scanner can only register one sale.

Say this to supervisor: "In the database, room inventory is governed by transactional ACID guarantees. We utilize PostgreSQL row locks (`SELECT ... FOR UPDATE`) or conditional constraints (`allocated_beds < capacity`) during the reservation transaction. If two students submit a booking request simultaneously, the first transaction commits and increments occupancy, causing the second to fail validation and receive a 409 Conflict."

Trap to avoid: Thinking JavaScript variables handle concurrency. Concurrency MUST be handled at the database transaction layer.

Q10: "What is your testing strategy? How do we know your code works?"

How to think about it: Before flying an airplane, engineers test the engines in a wind tunnel, not with passengers on board.

Say this to supervisor: "We built an automated test suite with over 30 unit and integration tests using Node's native test runner and TypeScript (`npm run test`). We test: (1) GPA calculation and Senate academic honors classification; (2) Exam clearance thresholds under varying fee balances; (3) M-Pesa STK payload formatting; (4) Cryptographic token forgery rejection; and (5) Token replay attack containment."

Trap to avoid: Saying "I tested it manually by clicking around." Automated tests prove you wrote rigorous software.

Q11: "What role does AI play in this project, and how is it integrated?"

How to think about it: Instead of having human academic advisors answer the same questions about prerequisites 500 times a day, the AI acts as an instant 24/7 university guide.

Say this to supervisor: "We integrated an Institutional AI Academic Advisor powered by NVIDIA NIM (`meta/llama-3.2-11b-vision-instruct`). Connected via our secure server route `/api/v1/ai/chat`, it acts as a digital advisor for students querying course prerequisites, academic policies, degree regulations, and graduation criteria, without exposing private student records to public LLM endpoints."

Trap to avoid: Don't claim the AI grading student exams. That raises ethical concerns. Emphasize that it is an academic guidance advisor.

Q12: "If this system is deployed to 30,000 students on exam day, how does it scale?"

How to think about it: Instead of one person answering 30,000 phone calls, you hire 10 call center reps and put the frequently asked questions on a giant billboard (caching).

Say this to supervisor: "Our decoupled architecture scales horizontally: (1) Next.js static assets and SSR pages can be distributed globally across Vercel Edge CDNs; (2) The Node.js Express backend can run in containerized Docker pods managed by Kubernetes behind a load balancer; (3) PostgreSQL connection pooling (via Supabase PgBouncer) prevents database exhaustion during traffic spikes; and (4) Static course catalog queries are cached to minimize direct database reads."

Trap to avoid: Saying "We will just buy a bigger server." Cloud software scales horizontally (more instances), not just vertically.

5. The Live Defense Walkthrough Script (Click-by-Click)

When presenting live, sit confidently, open your laptop, and follow this exact demonstration narrative:

Phase	What You Click on the Screen	What You Say Out Loud to the Supervisor
Phase 1: Architecture	Open browser at `http://localhost:3000`.	*"Good morning, Professor. This is the University Administration Management System. As you can see, the application is responsive, adhering to institutional branding. Behind this is our decoupled Next.js and Express Node.js architecture with PostgreSQL."*
Phase 2: Security & Login	Click **Sign In** to go to `/login`. Click the **Student (Faith Wanjiku)** demo button. Click **Authenticate & Sign In**.	*"Notice our authentication model. In compliance with your feedback, passwords are never stored in plaintext—we use salted scrypt hashing. The session token is transmitted in an HttpOnly cookie with SameSite protection, eliminating localStorage XSS vulnerabilities."*
Phase 3: Student Clearance	Navigate to **Exam Clearance & Cards** (`/dashboard/exams`).	*"Here is the student portal. If the student attempts to generate their Exam Card, our Institutional Policy Engine checks whether they have met both the 100% fee threshold and 75% attendance threshold. If uncleared, the card is locked with clear policy guidance."*
Phase 4: M-Pesa Finance	Navigate to **Fees & Finance** (`/dashboard/finance`). Initiate a simulated M-Pesa payment.	*"To clear arrears, students initiate real-time payments through our Safaricom Daraja integration. We engineered the webhook callback with strict idempotency so duplicate network retries never double-credit the student ledger."*
Phase 5: Lecturer Grading	Log out. Click **Lecturer** demo button and log in. Navigate to **Examinations & Grades** (`/dashboard/grades`).	*"Now I have logged in as Faculty. Notice that the navigation dynamically adapts based on RBAC. The lecturer enters Continuous Assessment (30%) and Final Exam (70%) marks. Our GPA engine automatically computes weighted credit averages."*
Phase 6: Senate Approval	Log out. Click **Registrar** demo button and log in. Show the approval workflow.	*"The Registrar has final Senate authority. Once verified, the Registrar executes Senate approval, permanently locking the marks from unauthorized tampering and generating verified institutional transcripts."*
Phase 7: AI Advisor & Audit Logs	Log in as **Super Admin**. Show **AI Advisor** (`/dashboard/ai`) and **Audit Logs** (`/dashboard/audit`).	*"Finally, as Super Admin, we have our AI Institutional Advisor powered by NVIDIA NIM for academic guidance, and our immutable Audit Trail where every financial and academic transaction is recorded for compliance."*